Technical Report — Autonomous Production Choke Controller (Problem 3A)

Honeywell Hackathon · Single naturally flowing oil well · Control interval Ts = 1 hr

---

1. Problem and approach

Set the production choke (0–100 %) once per hour to track a target oil rate while never violating the pressure operating envelope (lower bounds on wellhead, flowline and bottomhole pressure) or the ±5 %/step choke ramp limit. If a target is physically unsafe, line out at the maximum achievable safe rate rather than chase it.

Our solution is a **model-based predictive controller that is safe by construction**. We identify a first-order dynamic model of the well from the provided step test, then, every interval, search the reachable choke moves, predict each one's effect on all pressures, and **only ever command a move we have proven keeps the well inside the envelope**. The same decision rule tracks reachable targets tightly and refuses unreachable ones gracefully.

2. Data and open-loop analysis

The dataset is a 120-hour open-loop step test with the choke stepped 30 → 40 → 55 → 45 → 65 %. Two structural facts drive the whole design:

- The response is **first-order** with a time constant of roughly 5 hr on oil rate (slower on pressures), with mild measurement noise.
- **Opening the choke raises oil rate but lowers all three pressures.** The safety constraints are therefore **lower bounds**, and the well is pushed toward them precisely when we chase higher production — the core tension the controller must manage.

3. Dynamic model identification

For each output y ∈ {Q, WHP, FLP, BHP} we fit an ARX(1) model relating the choke *u* to the output one step ahead:

$$y[k+1] = a\, y[k] + (1-a)\,(b_0 + b_1\, u[k])$$

This is linear in the parameters, so we recover (a, b0, b1) by ordinary least squares (regress *y[k+1]* on *[y[k], 1, u[k]]*) and read off the physically meaningful quantities: steady-state gain $b_1 = dy_{cc}/du$, intercept b_0, and time constant $\tau = -Ts/\ln a$.

Output	gain b_1	b_0	τ (hr)	noise σ	free-run RMSE
Q (bbl/hr)	**+1.815**	39.31	5.15	0.74	1.5 % of span
WHP (psi)	**−1.586**	319.39	8.52	0.67	1.8 % of span
FLP (psi)	**−0.986**	219.27	6.76	0.52	2.4 % of span
BHP (psi)	**−8.383**	3417.35	12.96	2.95	1.9 % of span

The signs confirm the physics (choke up → rate up, pressures down). Validation is a genuine **free-run**: the model is initialised once and simulated open-loop under the recorded choke sequence using only its own predictions — never the measured output — and still matches the data to 1.5–2.4 % of signal span (`results/model_validation.png`). The one-step residual standard deviations set the noise level of the plant simulator so closed-loop tests see realistic scatter.

4. Operating envelope

The brief specifies no numeric envelope, so we adopt a **documented assumption**, chosen so the maximum safe rate sits high in the observed range and Scenario C is unambiguously infeasible. All limits live in one module (`envelope.py`):

Constraint	Value
WHP_min	215 psi
FLP_min	150 psi
BHP_min	2850 psi
Choke range	0–100 %
Ramp limit	±5 %/step

With the identified maps, **WHP is the first binding constraint**: it reaches 215 psi at choke ≈ 65.8 %,

giving a hard maximum safe rate of **≈ 159 bbl/hr**. When the official envelope/simulator is released, only this one file changes.

Per the brief, **wellhead temperature (WHT) and annulus pressure (AP)** are part of a complete production operating envelope but are *informational* here — not active constraints in this challenge — so they are not modelled or bounded. The envelope module is structured so they could be added as additional bounded outputs without touching the controller.

5. Controller design

A simplified MPC by brute-force candidate evaluation (explicitly permitted by the brief). Each interval:

1. **Read** current Q, WHP, FLP, BHP and choke position.
2. **Enumerate** candidate next positions: choke ± up to 5 %, on a 0.5 % grid, clipped to [0, 100] — every candidate already respects the ramp limit.
3. **Predict & screen for safety.** For each candidate, simulate the pressure trajectory over a settling horizon *and* compute its steady state with the ARX models. **Reject** the candidate if any pressure would fall below its bound plus a safety margin, at any predicted step or at steady state.
4. **Optimise.** Among the safe candidates, minimise `(Q_ss − target)² + λ·(Δchoke)²`.
5. **Infeasible target / recovery.** Because every safe candidate then has `Q_ss < target`, the cost automatically selects the **highest safe rate**. If no candidate is safe (e.g. the plant starts outside the envelope), the controller backs the choke off to the move that best restores the pressures.

Why this is safe by construction. A stable first-order response moves *monotonically* from the current output to its steady state. So if the current pressure is inside the envelope and the candidate's steady-state pressure is inside it (with margin), every intermediate value is too. Proving steady-state feasibility therefore proves trajectory feasibility — the controller never commits to a move it has not shown to be safe. The safety margin is sized to the fitted process noise (≈ 4σ on the binding pressure), which is what turns "predicted-safe" into "measured 0 violations" despite sensor scatter.

6. Results

Three required scenarios, run closed-loop against the simulator (`python main.py`, reproduced in `results/metrics.json`):

Scenario	Target	Settled rate	Settled choke	Steady-state error	Min WHP	Violations
A Startup → Target	130	129.9	50.0 %	−0.11 bbl/hr	238.8 psi	**0**
B Target Tracking	100 → 150	149.7	61.0 %	−0.34 bbl/hr	221.9 psi	**0**
C Infeasible	185	154.9	64.0 %	(infeasible)	216.7 psi	**0**

- **A** brings the well from startup (≈ 90 bbl/hr) to 130 and holds it, tracking to 0.11 bbl/hr with 18.8 psi of pressure headroom to spare.
- **B** tracks a mid-run target step 100 → 150, settling to 0.34 bbl/hr while WHP lines out at 221.9 psi — inside the envelope throughout.
- **C** requests 185 bbl/hr, which would require choke ≈ 80 % and drive WHP to ≈ 192 psi (well below the 215 floor). The controller **refuses**, settling at the maximum safe rate (154.9 bbl/hr) with WHP held at 216.7 psi and **zero violations**.

Each scenario produces the required six-trend plot (Target Q, Actual Q, WHP, FLP, BHP, Choke) in `results/scenario_{A,B,C}.png`.

7. Safety performance vs. a naive baseline

To quantify the value of the safety logic, the same scenarios were run with a naive proportional controller that chases the oil-rate target with no envelope awareness:

Scenario	Safety-first MPC violations	Naive baseline violations
A	0	0
B	0	0
C	**0**	**168** (WHP 57, FLP 56, BHP 55)

On the feasible targets (A, B) both reach the setpoint. On the infeasible target (C) the naive controller drives the choke past 100 %, holding the well ~65 psi below the WHP floor for the entire run — 168 constraint breaches — to gain just ~30 bbl/hr of unsafe, unsustainable production. The safety-first controller gives that up by design. `results/baseline_comparison_C.png` shows the contrast.

8. Robustness — stressing the controller beyond its model

"Zero violations when the plant equals the model" is only half a claim, so we ran a 22-case stress study (`python robustness.py`, results in `results/robustness_metrics.json`) where the controller **keeps its identified model** while the true plant is altered:

Stress family	Cases	Violations	Outcome
Measurement noise 1× / 2× (3 seeds each)	6	**0**	margins absorb the scatter
Measurement noise 4×, margin unaware	3	2 (Scenario C, dips ≤ 2.2 psi)	see below
Measurement noise 4×, margin sized to true noise	3	**0**	settles lower (143 vs 155 bbl/hr)
Plant gains ±20 % (re-anchored at startup)	6	**0**	production sacrificed, never safety
Stress: pressures 20 % steeper + dynamics 30 % slower	3	**0**	settles at 136 bbl/hr, margin 0.2 psi
Unmeasured −8 psi WHP disturbance at t = 60 hr	1	**0**	recovers on feedback alone

Two findings worth stating honestly:

- **Under model mismatch the controller fails in the right direction.** With the true pressure gains 20 % steeper than the model believes, feedback pulls the choke back as the measured WHP approaches the floor: the well settles at a *lower* rate (135.8–156.8 bbl/hr depending on case) with **zero violations**. Model error costs production, never safety.
- **The safety margin is an explicit dial, and the one failure shows why.** At 4× the identified noise with the margin still sized for 1×, Scenario C grazes the floor twice (≤ 2.2 psi). Re-sizing the margin to the true noise — no logic change, the margin already scales with the controller's noise estimate — restores zero violations at every noise level, trading ~12 bbl/hr of throughput for it. That is the intended contract: uncertainty is paid for in production, not in envelope breaches.

`results/robustness_margins.png` shows the tightest margin per case; `results/robustness_disturbance.png` shows the disturbance recovery.

9. Lessons learned & limitations

- **The binding constraint is a lower pressure bound, not an upper rate bound** — recognising that inverted the intuition and made WHP the design driver.
- **Monotonicity is the key that makes a cheap MPC provably safe.** A short brute-force search plus a steady-state feasibility check is enough to guarantee the envelope; no heavy optimiser is required.
- **The safety margin trades a little production for a lot of robustness.** At ~4σ, measured violations are zero at the cost of settling ~4 bbl/hr below the theoretical hard limit — a deliberate, adjustable choice.
- **Limitations.** The plant model is first-order and linear. The brief's simulator assumptions (single naturally flowing well, no gas lift/ESP, constant reservoir properties, constant GOR and water cut) keep that valid for this challenge, but a real well would add mild nonlinearity, gas-coning and slugging the ARX model will not capture; the envelope numbers are also our assumption. The controller design is agnostic to all of this: `simulator.WellSimulator` shares the official `.step()` signature, so the real simulator drops in with a one-line change, and the envelope is a single editable module.

10. Reproducing

```
pip install -r requirements.txt
python main.py        # prints model params, runs A/B/C, asserts 0 violations
python robustness.py  # 22-case stress study (noise, mismatch, disturbance)
streamlit run app.py  # live dashboard: scenarios, per-step MPC diagnostics
```

All figures and `metrics.json` are regenerated in `results/`. The narrative walk-through is in `notebook/choke_control_solution.ipynb`.